Phase 5 – three tickets in a row: why we need central logs

Project 2 – Acme Analytics Linux platform · ticket #8 (8 Nov) → ticket #9 (9 Nov) → ticket #10 (Step 5.4, 11 Nov) · short version on this page, details on the next pages

Ticket #8 · Step 5.2

"Black Friday – the stores still show old prices"

PROBLEM

The price script on **app01** should cut all prices by 30 % at 02:00. The store screens still show old prices. The script writes **no log lines** – nobody knows if it ran or why it failed.

SOLUTION

Give the script a diary with `logger`: "started" · "FAILED + reason" · "done: 3 prices changed".

KEY COMMANDS

```
logger -t price-update "started"
logger -t price-update -p user.err "FAILED: ... missing"
journalctl -t price-update   # did it run? did it fail? why?
```

REAL COMPANY

The script changes the prices in the database; the store screens read them from there.

Lesson: a script without log lines fails silently. Now it has a log – **but only on app01**.

→ Full solution: page 2

Ticket #9 · Day 16

"The web shop froze for 3–4 minutes on Saturday"

PROBLEM

A freeze can come from 4 places – each has its own log, on a different server.

KEY COMMANDS (one per place)

```
# 1 the app (app01)
journalctl -u nginx --since "10 min ago"
# 2 the whole server (app01)
journalctl --list-boots
journalctl -k -b -3 | grep -i -E "lockup|stuck|watchdog|reset"
# 3 the database (db01)
tail /var/log/postgresql/postgresql-14-main.log
# 4 the network (app01 → db01)
PGCONNECT_TIMEOUT=5 psql -h db01 ...  +  ping db01
```

SOLUTION

Place 2 was the cause: "CPU stuck for 160 s" at 19:22 – app01 got no CPU (host PC sleep; in a company: overloaded virtualization host).

Lesson: the answer was in the logs, but **spread over app01 and db01**, and frozen app01 could not be read.

→ Full solution: pages 3–4

Ticket #10 · Step 5.4

"Logs are spread over every server"

PROBLEM

Logs live only on the server that wrote them. Lost or unreadable when:

1. the server is **frozen** (ticket #9)
2. an **attacker** breaks in and deletes them to hide the attack
3. an **insider** (an employee) deletes them on purpose, to hide a mistake or misuse
4. someone deletes them **by mistake** (wrong `rm`, cleanup script)
5. the server is **gone** (disk broken, VM deleted)

SOLUTION

app01 + db01 send every log line with **rsyslog** over **TCP 514** to **stor01**. A **queue** keeps lines while stor01 is down and sends them later.

KEY COMMANDS

```
sudo ss -tlnp | grep 514          # stor01 listens
logger -t ticket10 "hello"        # on app01
sudo tail /var/log/remote/app01/ticket10.log  # on stor01
```

Lesson: logs in **two places** – on the server (5.5) and on stor01 (5.4). **Phase 6 auditd** adds: *who* deleted or changed a log file.

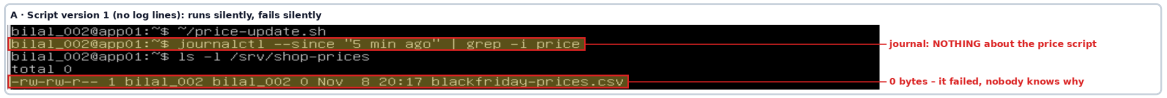
→ Plan and commands: page 5

Page 2 · Ticket #8 – full solution (Step 5.2, Day 15)

Acme Fashion (made-up name) · Markus (Sales): "Did the price script run, did it fail, and why?" · server: app01 · details: docs/progress/day-15-2026-11-08.md, section 8

A · The script as it was – silent

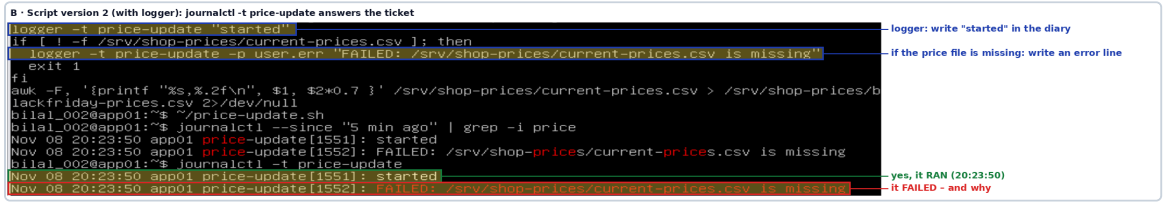
```
awk -F, '{ printf "%s,%.2f\n", $1, $2*0.7 }' current-prices.csv > blackfriday-
prices.csv 2>/dev/null
```



Journal empty, result file 0 bytes → it failed, but **no reason anywhere**.

B · Same script with a diary (logger) – run without the price file

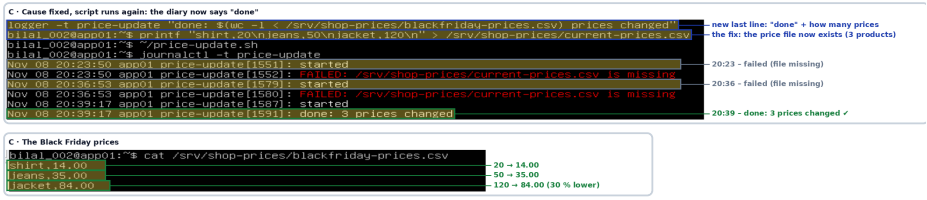
```
logger -t price-update "started"
if [ ! -f /srv/shop-prices/current-prices.csv ]; then
  logger -t price-update -p user.err "FAILED: /srv/shop-prices/current-prices.csv is
missing"; exit 1
fi
awk ... > blackfriday-prices.csv
logger -t price-update "done: $(wc -l < blackfriday-prices.csv) prices changed"
```



`journalctl -t price-update` → **started** (it ran) + **FAILED: current-prices.csv is missing** in red (why).

C · Fix the cause (put the price file back) and run again

```
printf "shirt,20\njeans,50\njacket,120\n" > /srv/shop-
prices/current-prices.csv
~/price-update.sh
journalctl -t price-update
cat /srv/shop-prices/blackfriday-prices.csv
```



done: 3 prices changed · 20 → 14 · 50 → 35 · 120 → 84 (30 % lower).

Answer to Markus (GLPI, solved): the script ran at night but stopped because the price file was missing; it wrote nothing, so nobody could see it. Now it writes started / FAILED + reason / done, and `journalctl -t price-update` answers the question in one command.

Words: `-t` = name tag of the lines · `-p user.err` = mark as error (red) · `exit 1` = stop with "failed".

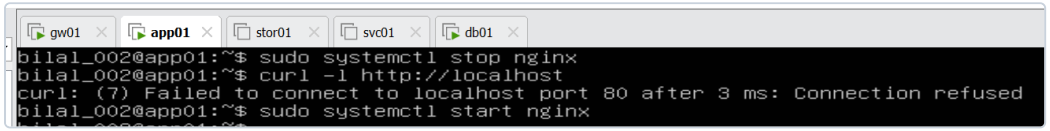
Page 3 · Ticket #9 – full solution, part 1 (Day 16)

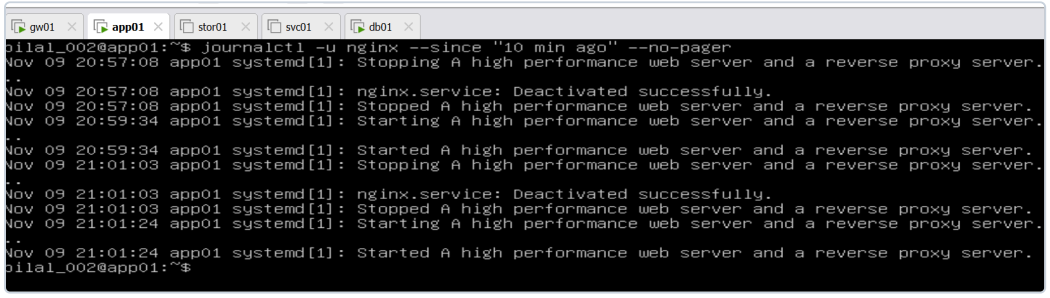
Sara: "The web shop froze for 3–4 minutes on Saturday, then worked again." · shop = nginx on app01 (.10) + PostgreSQL on db01 (.30) · details: docs/progress/day-16-2026-11-09.md, section ③

Place	Where	Command	The line that answers	Result
① the app	app01	`journalctl -u nginx`	Stopping → Stopped → Started, with times	shown
② the whole server	app01	`journalctl -k -b -3 | grep ...`	**soft lockup – CPU#1 stuck for 160s** at 19:22	**the real cause**
③ the database	db01	`tail postgresql-14-main.log`	duration: 10088 ms statement: ...	shown
④ the network	app01 → db01	`psql` + `ping`	timeout expired · ping 0 % loss	shown

① The app – stop nginx, what Sara feels, then nginx's own log

```
sudo systemctl stop nginx ; curl -I http://localhost ; sudo systemctl start
nginx
journalctl -u nginx --since "10 min ago" --no-pager
```

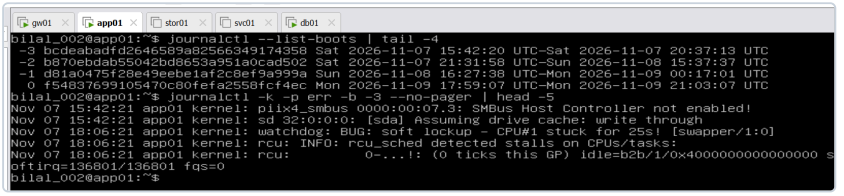


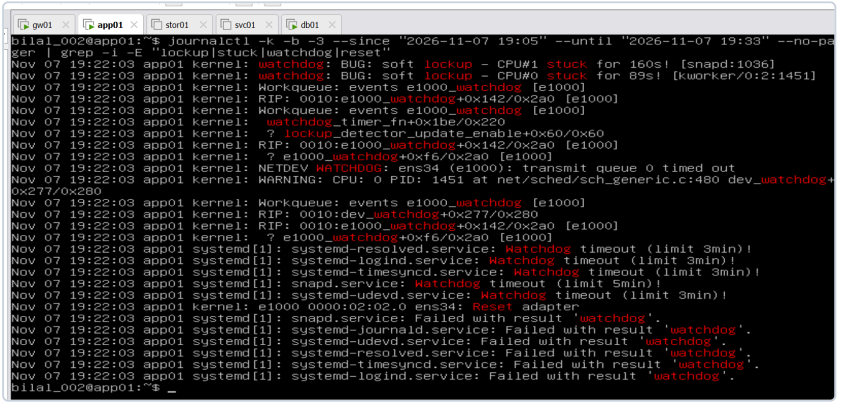


> "Connection refused" = the app is down. The log shows exactly when it stopped and started.

② The whole server – Saturday's run is now -3 (numbers move back after each restart)

```
journalctl --list-boots | tail -4
journalctl -k -b -3 --since "2026-11-07 19:05" --until "2026-11-07 19:33" |
grep -i -E "lockup|stuck|watchdog|reset"
```





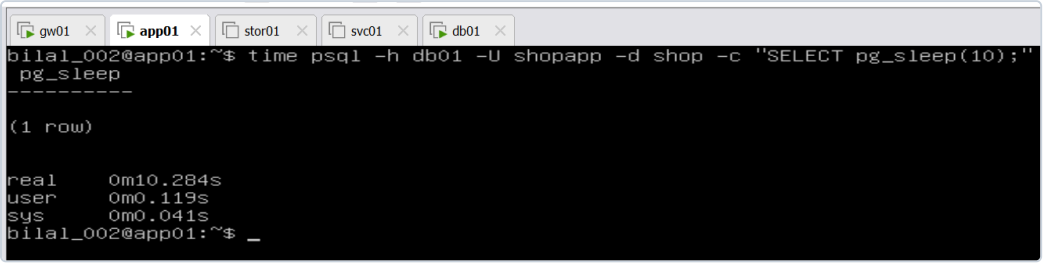
> 19:22 · **CPU stuck for 160s / 89s** · NETDEV WATCHDOG · Reset adapter → app01 got no CPU for minutes.

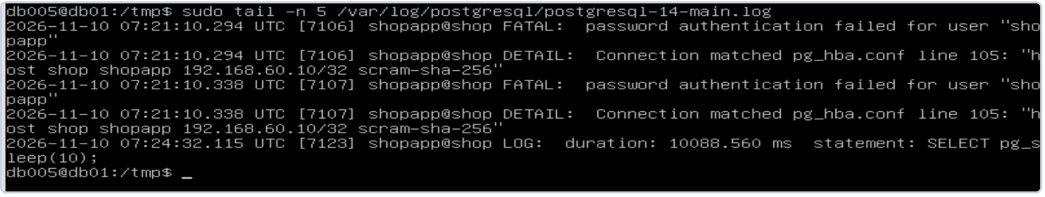
Page 4 · Ticket #9 – full solution, part 2

The database and the network – shown to rule them out

③ The database – a 10-second query from app01, then db01's slow-query log

```
time psql -h db01 -U shopapp -d shop -c "SELECT pg_sleep(10);"     # on app01
sudo tail -n 3 /var/log/postgresql/postgresql-14-main.log           # on
db01
```

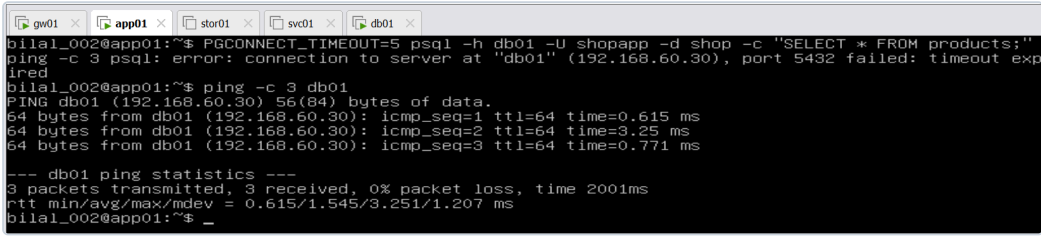


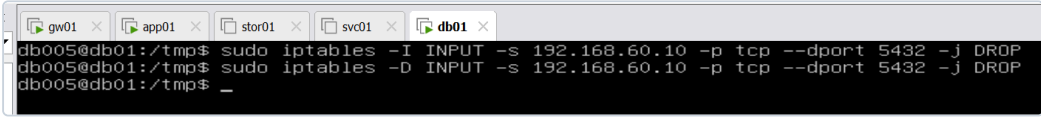


> app01 waited 10.28 s · db01 logged **duration: 10088 ms** + the query (because `log_min_duration_statement = 2000` logs every query over 2 s).

④ The network – block port 5432 for a moment

```
sudo iptables -I INPUT -s 192.168.60.10 -p tcp --dport 5432 -j DROP   # on
db01
PGCONNECT_TIMEOUT=5 psql -h db01 -U shopapp -d shop -c "SELECT * FROM
products;"   # on app01
ping -c 3 db01                                                          # on
app01
sudo iptables -D INPUT -s 192.168.60.10 -p tcp --dport 5432 -j DROP   # on
db01 — remove
```





> "timeout expired" while **ping 0 % loss** → "ping works" ≠ "the service works".

Answer to Sara (solved): the shop and the database were fine. At 19:22 the whole app01 got no CPU for about 2–3 minutes (kernel: *soft lockup – CPU stuck for 160s*). Cause: the virtualization host (lab: the host PC went to sleep) – in a company this goes to the virtualization team. **What it showed for the next ticket:** places ① ② were on app01, ③ on db01 – the admin had to log in to each server, and could not read app01 while it was frozen → ticket #10.

Page 5 · Ticket #10 – central logs on stor01 (Step 5.4) · ✔ solved

Sara: "Collect the logs of all servers in one place, so we can read them when a server is frozen or broken – and nobody can delete the copy." · status: solved 11 Nov – all screenshots in Day 17 (docs/progress/day-17-2026-11-11.md)

Layout after Step 5.4

app01 (.10) ─ TCP 514 ─▶ **stor01 (.20)** ◀─ TCP 514 ─ db01 (.30)

stor01 keeps: `/var/log/remote/<server>/<program>.log`

e.g. `app01/price-update.log` (ticket #8) · `app01/kernel.log` (ticket #9 ②) · `db01/postgres.log` (ticket #9 ③)

A · stor01 receives

```
echo "192.168.60.30 db01" | sudo tee -a /etc/hosts
sudo mkdir -p /var/log/remote && sudo chown syslog:adm /var/log/remote
sudo nano /etc/rsyslog.d/10-receive.conf      # imtcp port 514 + template + ruleset
sudo rsyslogd -N1 && sudo systemctl restart rsyslog
sudo ss -tlnp | grep 514                      # GOOD: 0.0.0.0:514 rsyslogd
```

B · app01 and db01 send (with a queue)

```
sudo nano /etc/rsyslog.d/90-forward.conf      # omfwd to 192.168.60.20:514, queue on disk
sudo rsyslogd -N1 && sudo systemctl restart rsyslog
```

C · Test

```
logger -t ticket10 "hello from app01"          # on app01 (and db01)
~/price-update.sh                              # ticket #8's script, on app01
sudo ls /var/log/remote/ ; sudo tail /var/log/remote/app01/price-update.log   # on stor01
```

D · Failure test – stor01 down for a while

```
sudo systemctl stop rsyslog                    # on stor01
logger -t ticket10 "sent while stor01 was down" # on app01 – waits in the queue
sudo systemctl start rsyslog                   # on stor01 – the waiting line arrives
```

Who can delete logs – and what protects them

Who / what	Protection
Attacker breaks in to app01 and deletes logs to hide the attack	copy on stor01 – the attacker is not logged in there
Insider (employee) deletes logs on purpose to hide a mistake or misuse	copy on stor01 (only admins of stor01 can touch it) + **auditd** (Phase 6)
Deleted **by mistake** (wrong rm, cleanup script)	copy on stor01
Server **frozen** or **gone**	copy on stor01
stor01 down for a while	queue on app01/db01, sent later

auditd (Phase 6, coming): the Linux "security camera" for files. A rule like `-w /var/log/ -p wa` writes a line every time someone **writes to or deletes** a log file – with the user, the time and the command. So the copy on stor01 shows *what* was in the log, and auditd shows *who* touched it. Companies need both for audits (ISO 27001, BSI).

Limit: a line a server never got to send (the last seconds before a hard crash) is missing everywhere. **Rule:** keep logs in two places – on the server (Step 5.5, persistent journal) and on stor01 (Step 5.4).

E · ✔ Ticket #10 solved. Proof on stor01: app01 + db01 test lines · ticket #8 price script (started / done: 3 prices changed) · ticket #9 slow-query line (duration: 3074 ms) · queue test: line written while stor01 was down arrived later with its original time.